Lab 5: Overview of UI Framework: Yew

Using Yew to Build and Develop a Client-side Application with Rust:

Introduction to Yew:

- Yew is a modern Rust framework for creating multi-threaded front-end web apps with WebAssembly.
- It features a component-based framework which makes it easy to create interactive user interfaces.
- It has great performance by minimizing DOM API calls and by helping developers easily offload processing to background web workers.
- It supports JavaScript interoperability, allowing developers to leverage NPM packages and integrate with existing JavaScript applications.
- Yew is a framework that takes many of the concepts of the Elm programming language and implements them into the Rust programming language.
- All Elm programs follow a powerful and opinionated MVC style and Yew adopts this style to make it possible to do Reactive Functional Programming in Rust.
- The Yew Framework also has a powerful macro system which allows the user to build HTML interfaces directly inside of Rust in much the same way that you would in a framework like React or Elm.

Resources:

- Yew official site and documentation: https://yew.rs/docs/getting-started/introduction
- The Yew Crate: https://crates.io/crates/yew
- More examples of Yew applications that you can test and learn from can be found here: https://yew.rs/docs/getting-started/examples
- Currently, the examples below has no styles and is not very appealing. However Trunk can be used to add style sheets to your application. The link to Trunk to use for styling is: https://trunkrs.dev/assets/
- The yew example below was referenced from https://yew.rs/docs/tutorial .

Prerequisites:

- Before we begin our web application using Yew, we must first install Trunk.
- Trunk is a WASM web application bundler for Rust.
- We must also add the wasm build target.
- To install trunk and add the wasm build target, we run the following in the command line:

```
$ cargo install trunk --version 0.21.7

$ rustup target add wasm32-unknown-unknown
```

- After that we can create a new rust project to start building our web application:

```
$ cargo new yew-app
$ cd yew-app

$ cargo run
```

Getting started:

- First let us add the yew dependency to the cargo.toml file.
- We should add "csr" to specify we want to enable Client-Side Rendering support and Renderer.

```
[package]
name = "yew-app"
version = "0.1.0"
edition = "2021"

[dependencies]
yew = { version = "0.21", features = ["csr"] }
```

- Next we should add the following code to main.rs to create a simple Hello World web application:

```
use yew::prelude::*;

#[function_component(App)]
fn app() -> Html {
    html! {
        <h1>{ "Hello World" }</h1>
    }
}

fn main() {
    yew::Renderer::<App>::new().render();
}
```

- We also need to create an index.html file in the project directory (in this example: yew-app).
- index.html is an essential file needed to create your application since it serves as the home page for the application.
- When used with Yew, we will mainly use this file to put html code in the <head>. Examples of code that goes in the head include:
    - title of webpage (i.e. <title>).
    - Styling and CSS
    - Linking (i.e. <link>)
- The following code should be added to index.html:

```
<!doctype html>
<html lang="en">
      <head></head>
      <body></body>
</html>
```

- Finally we can run our web application using the following command in the command line:

```
$ trunk serve --open
```

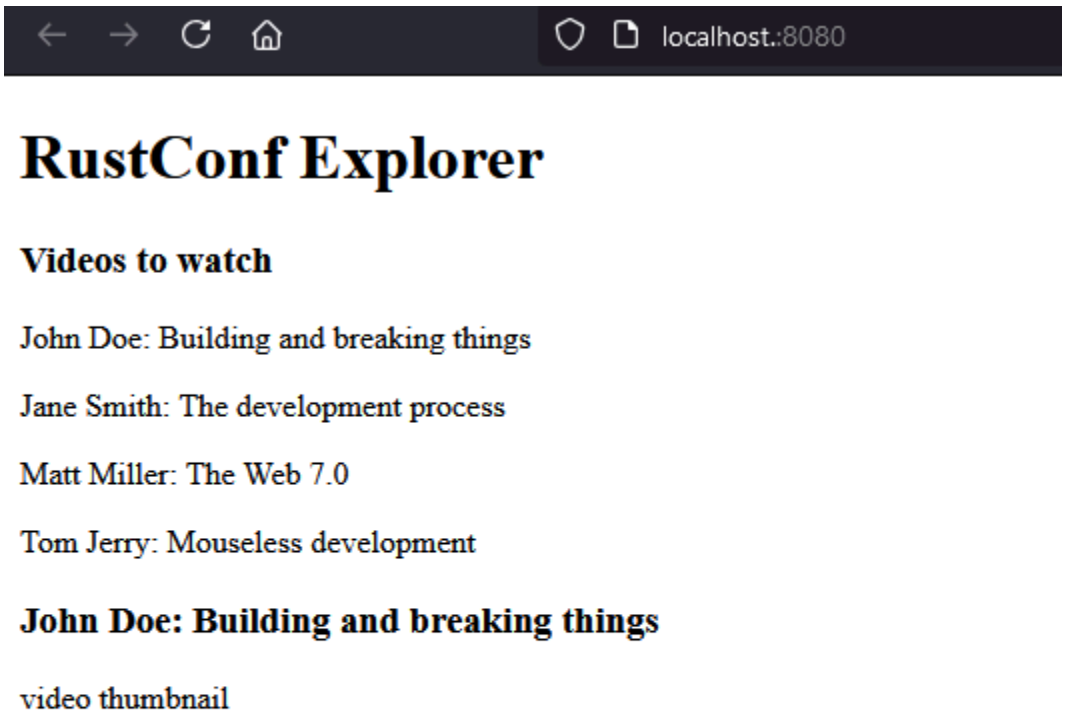
- Your application should be running on http://localhost:8080/ or http://127.0.0.1:8080/ in a web browser.
- Hello World should be written on the web page.

Using HTML:

- Using html is simple. We can simply write our html code in the html!{ … }.
- However there are some differences between normal html and the html!{ … } that we are using here in rust:
  - Expressions must be wrapped in curly braces and quotes ({" "}). (As seen below).
  - There must only be one root node. If you want to have multiple elements without wrapping them in a container, an empty tag/fragment (<> ... </>) is used
  - Elements must be closed properly.
- Here is an example of how to apply html code to our program:

```
use yew::prelude::*;

#[function_component(App)]
fn app() -> Html {
-    html! {
-        <h1>{ "Hello World" }</h1>
-    }
+    html! {
+        <>
+            <h1>{ "RustConf Explorer" }</h1>
+            <div>
+                <h3>{"Videos to watch"}</h3>
+                <p>{ "John Doe: Building and breaking things" }</p>
+                <p>{ "Jane Smith: The development process" }</p>
+                <p>{ "Matt Miller: The Web 7.0" }</p>
+                <p>{ "Tom Jerry: Mouseless development" }</p>
+            </div>
+            <div>
+                <h3>{ "John Doe: Building and breaking things" }</h3>
+                <img
src="https://via.placeholder.com/640x360.png?text=Video+Player+Placeholder"
alt="video thumbnail" />
+            </div>
+        </>
+    }
}

fn main() {
    yew::Renderer::<App>::new().render();
}
```

- Running the program should result in the following output on the webpage:



Rust constructs in markup:

- Instead of hardcoding all the code in the html, we can utilize rust constructs in order to make the html code more dynamic.
- In the code sample below, we used a vector to represent the video titles we want to appear on the webpage.
- The vector is then converted to html code and used in the html.
- This lets our html code be dynamic since we can change the size and values within the vector without having to modify the html code.

```
use yew::prelude::*;

+ struct Video {
+    id: usize,
+    title: String,
+    speaker: String,
+    url: String,
+ }
```

```
#[function_component(App)]
fn app() -> Html {
+    let videos = vec![
+        Video {
+            id: 1,
+            title: "Building and breaking things".to_string(),
+            speaker: "John Doe".to_string(),
+            url: "https://youtu.be/PsaFVLr8t4E".to_string(),
+        },
+        Video {
+            id: 2,
+            title: "The development process".to_string(),
+            speaker: "Jane Smith".to_string(),
+            url: "https://youtu.be/PsaFVLr8t4E".to_string(),
+        },
+        Video {
+            id: 3,
+            title: "The Web 7.0".to_string(),
+            speaker: "Matt Miller".to_string(),
+            url: "https://youtu.be/PsaFVLr8t4E".to_string(),
+        },
+        Video {
+            id: 4,
+            title: "Mouseless development".to_string(),
+            speaker: "Tom Jerry".to_string(),
+            url: "https://youtu.be/PsaFVLr8t4E".to_string(),
+        },
+    ];

+   let videos = videos.iter().map(|video| html! {
+       <p key={video.id}>{format!("{}: {}", video.speaker, video.title)}</p>
+   }).collect::<Html>();

    html! {
        <>
            <h1>{ "RustConf Explorer" }</h1>
            <div>
                <h3>{ "Videos to watch" }</h3>
-               <p>{ "John Doe: Building and breaking things" }</p>
-               <p>{ "Jane Smith: The development process" }</p>
-               <p>{ "Matt Miller: The Web 7.0" }</p>
-               <p>{ "Tom Jerry: Mouseless development" }</p>
+               { videos }
            </div>
            // ...
        </>
    }
}

fn main() {
```

```
    yew::Renderer::<App>::new().render();
}
```

Components:

- Components are the building blocks that we use to build our applications.
- Components are used to allow our code to be reusable and keep it generic.
- There are two types of components: function and struct components.
- In the following example, we show how to use video lists as its own function component rather than using it in the App function component.
- This makes our code cleaner and reusable.

```
use yew::prelude::*;

+ #[derive(Clone, PartialEq)]
struct Video {
    id: usize,
    title: String,
    speaker: String,
    url: String,
}

+ #[derive(Properties, PartialEq)]
+ struct VideosListProps {
+    videos: Vec<Video>,
+ }

+ #[function_component(VideosList)]
+ fn videos_list(VideosListProps { videos }: &VideosListProps) -> Html {
+    videos.iter().map(|video| html! {
+        <p key={video.id}>{format!("{}: {}", video.speaker,
video.title)}</p>
+    }).collect()
+ }

#[function_component(App)]
fn app() -> Html {
    // ...
-    let videos = videos.iter().map(|video| html! {
-        <p key={video.id}>{format!("{}: {}", video.speaker,
video.title)}</p>
-    }).collect::<Html>();
-
    html! {
        <>
            <h1>{ "RustConf Explorer" }</h1>
            <div>
                <h3>{"Videos to watch"}</h3>
-               { videos }
```

```
+                <VideosList videos={videos} />
            </div>
            // ...
        </>
    }
}

fn main() {
    yew::Renderer::<App>::new().render();
}
```

Making an Interactive Web Application:

- To make an interactive web application, we need to use Callbacks.
- We use Callbacks to pass event handlers between components, which makes the web application interactive.
- In this example, the VideoList component needs to use a Callback to notify when a video is selected.
- VideoList component will "emit" the video to the Callback.
- We created a new component VideoDetails to display the video details when clicked.
- App component is then modified to display the video details when clicked.
- We use use_state in the App component. use_state is called a "hook". A hook is a function that lets you store state and perform side-effects.
- After implementing the following changes, if you click on the names of the videos in the webpage, the webpage should update accordingly.

```
// ...

#[derive(Properties, PartialEq)]
struct VideosListProps {
    videos: Vec<Video>,
+    on_click: Callback<Video>
}

- #[function_component(VideosList)]
- fn videos_list(VideosListProps { videos }: &VideosListProps) -> Html {
-    videos.iter().map(|video| html! {
-                    <p   key={video.id}>{format!("{}:   {}",   video.speaker,
video.title)}</p>
-    }).collect()
- }

+ #[function_component(VideosList)]
+ fn videos_list(VideosListProps { videos, on_click }: &VideosListProps) ->
Html {
+     let on_click = on_click.clone();
+     videos.iter().map(|video| {
+           let on_video_select = {
+           let on_click = on_click.clone();
```

```
+           let video = video.clone();
+           Callback::from(move |_| {
+                 on_click.emit(video.clone())
+           })
+           };
+     html! {
+           <p  key={video.id}  onclick={on_video_select}>{format!("{}:  {}",
video.speaker, video.title)}</p>
+     }
+     }).collect()
+ }


+ #[derive(Properties, PartialEq)]
+ struct VideosDetailsProps {
+    video: Video,
+ }

+ #[function_component(VideoDetails)]
+ fn video_details(VideosDetailsProps { video }: &VideosDetailsProps) -> Html
+ {
+    html! {
+        <div>
+            <h3>{ video.title.clone() }</h3>
+            <img
src="https://via.placeholder.com/640x360.png?text=Video+Player+Placeholder"
alt="video thumbnail" />
+        </div>
+    }
+ }

#[function_component(App)]
fn app() -> Html {
    // ...
+    let selected_video = use_state(|| None);

+    let on_video_select = {
+        let selected_video = selected_video.clone();
+        Callback::from(move |video: Video| {
+            selected_video.set(Some(video))
+        })
+    };

+    let details = selected_video.as_ref().map(|video| html! {
+        <VideoDetails video={video.clone()} />
+    });

    html! {
        <>
            <h1>{ "RustConf Explorer" }</h1>
```

```
            <div>
                <h3>{"Videos to watch"}</h3>
-               <VideosList videos={videos} />
+               <VideosList videos={videos}
on_click={on_video_select.clone()} />
            </div>
+            { for details }
-            <div>
-                <h3>{ "John Doe: Building and breaking things" }</h3>
-                <img
src="https://via.placeholder.com/640x360.png?text=Video+Player+Placeholder"
alt="video thumbnail" />
-            </div>
        </>
    }
}

fn main() {
    yew::Renderer::<App>::new().render();
}
```

Fetching data using external REST API:

- Usually data comes from an external API.
- In order to fetch the data from these external API, we will need to using the following crates:
    - gloo-net: For making the fetch call.
    - serde: Has derived features for de-serializing the JSON response.
    - wasm-bindgen-futures: For executing Rust Future as a Promise.

```
[dependencies]
yew = { version = "0.21", features = ["csr"] }
+ gloo-net = "0.2"
+ serde = { version = "1.0", features = ["derive"] }
+ wasm-bindgen-futures = "0.4"
```

- Finally we will modify the code to make the fetch request instead of using hardcoded data.

```
use yew::prelude::*;
+ use serde::Deserialize;
+ use gloo_net::http::Request;

- #[derive(Clone, PartialEq)]
+ #[derive(Clone, PartialEq, Deserialize)]
struct Video {
    id: usize,
    title: String,
    speaker: String,
    url: String,
}
```

```
// ...

#[function_component(App)]
fn app() -> Html {
-    let videos = vec![
-        // ...
-    ]
+     let videos = use_state(|| vec![]);
+     {
+     let videos = videos.clone();
+     use_effect_with((), move |_| {
+           let videos = videos.clone();
+           wasm_bindgen_futures::spawn_local(async move {
+                 let fetched_videos: Vec<Video> =
Request::get("/tutorial/data.json")
+                 .send()
+                 .await
+                 .unwrap()
+                 .json()
+                 .await
+                 .unwrap();
+                 videos.set(fetched_videos);
+           });
+           || ()
+     });
+     }

    // ...

    html! {
        <>
            <h1>{ "RustConf Explorer" }</h1>
            <div>
                <h3>{"Videos to watch"}</h3>
-                <VideosList videos={videos}
on_click={on_video_select.clone()} />
+                <VideosList videos={(*videos).clone()}
on_click={on_video_select.clone()} />
            </div>
            { for details }
        </>
    }
}
```

- In order to avoid problems with CORS (Cross-origin resource sharing), we will use a proxy server.

```
$ trunk serve --proxy-backend=https://yew.rs/tutorial
```

Simple Interactive Input Examples in Yew:

A button that accepts an argument:

- To accept an argument for callback functions, we need to use the ".emit" function.
- Below is a simple example of a counter function that accepts "input" as an argument, which determines the counter increment amount.

```
use yew::prelude::*;


// Program counts the number of times the button is pressed.
#[function_component(App)]
fn app() -> Html {
      // Variable to store the total value of the counter. Initialized at 0.
      let result = use_state(|| 0);

      // Recognizes click event and updates the result variable.
      let onclick = Callback::from({
      let result = result.clone();
      move |number|{
            // Adds input value to total.
            let value = *result + number;
            result.set(value);
      }
      });

      // Value we want to use as an argument. This is how much the counter
will increment by.
      let input = 1;


      // html code. Creates an "Add" button.
      html! {
      <>
            <input type="button" value="Add"
                  // We use emit to pass the argument to the onclick callback
function when the button is clicked.
                  onclick={let   onclick   =   onclick.clone();   move   |_|
onclick.emit(input)}
            />
            <p>{*result}</p>
      </>
      }
}
```

```
fn main() {
      yew::Renderer::<App>::new().render();
}
```

A button that recognizes keyboard input:

- Below is a simple example of a program that displays the last keyboard key you have pressed to the screen.

```
use yew::prelude::*;


// Program that displays the last button you have pressed.
#[function_component(App)]
fn app() -> Html {
      // Variable to store the last button you have pressed. Initialized
message is: "Press display then press any key on your keyboard."
      let result = use_state(|| "Press display then press any key on your
keyboard.".to_string());

      // Recognizes key press events and updates the result variable.
      let onkeydown = Callback::from({
      let result = result.clone();
      move |event:KeyboardEvent|{
            let value = event.key();
            result.set(value);
      }
      });

      // html code. Creates a "Display" button, a message that says "Last key
pressed:", and outputs the last key pressed.
      html! {
      <>
            <input type="button" value="Display"
                  // Calls the onkeydown callback function when a key is
pressed down.
                  onkeydown={onkeydown.clone()}
            />
            <p>{"Last key pressed:"}</p>
            // Displays the last key pressed to the screen.
            <p>{(*result).clone()}</p>
      </>
      }
}


fn main() {
      yew::Renderer::<App>::new().render();
}
```

Interactive Calculator

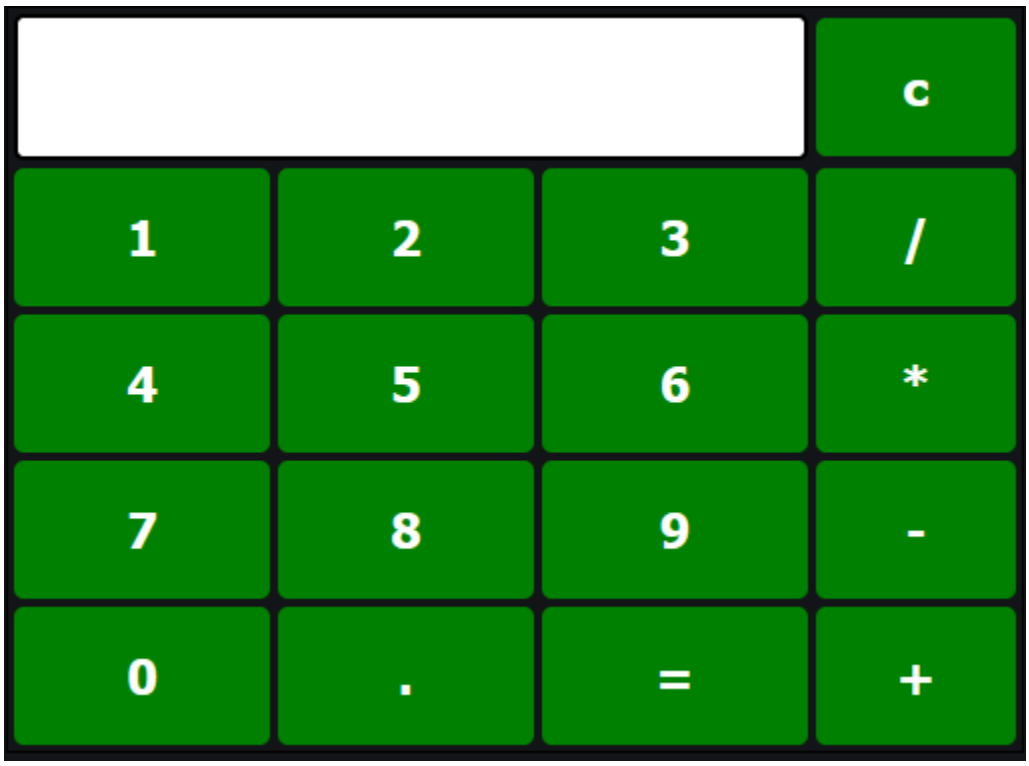
Consider the following HTML code:

```
<!DOCTYPE html>
<html>

<head>
        <title>JavaScript Calculator</title>
        <script src=
"https://cdnjs.cloudflare.com/ajax/libs/mathjs/10.6.4/math.js"
        integrity=
"sha512-BbVEDjbqdN3Eow8+empLMrJlxXRj5nEitiCAK5A1pUr66+jLVejo3PmjIaucRnjlB0P9R3rBUs3g5
jXc8ti+fQ=="
        crossorigin="anonymous"
        referrerpolicy="no-referrer"></script>
        <script src=
"https://cdnjs.cloudflare.com/ajax/libs/mathjs/10.6.4/math.min.js"
        integrity=
"sha512-iphNRh6dPbeuPGIrQbCdbBF/qcqadKWLa35YPVfMZMHBSI6PLJh1om2xCTWhpVpmUyb4IvVS9iYnn
YMkleVXLA=="
        crossorigin="anonymous"
        referrerpolicy="no-referrer"></script>

        <!-- For styling -->
        <style>
        table {
        border: 1px solid black;
        margin-left: auto;
        margin-right: auto;
        }

        input[type="button"] {
        width: 100%;
        padding: 20px 40px;
        background-color: green;
        color: white;
        font-size: 24px;
        font-weight: bold;
        border: none;
        border-radius: 5px;
        }

        input[type="text"] {
        padding: 20px 30px;
        font-size: 24px;
        font-weight: bold;
        border: none;
        border-radius: 5px;
        border: 2px solid black;
        }
        </style>
</head>
```

```
<body>

        <!-- Use Table to Create Calculator Structure Design -->
        <table id="calcu">
        <tr>
        <td colspan="3"><input type="text" id="result"></td>
        <td><input type="button" value="c" onclick="clr()" /> </td>
        </tr>
        <tr>
        <td><input type="button" value="1" onclick="dis('1')"
                        onkeydown="myFunction(event)"> </td>
        <td><input type="button" value="2" onclick="dis('2')"
                        onkeydown="myFunction(event)"> </td>
        <td><input type="button" value="3" onclick="dis('3')"
                        onkeydown="myFunction(event)"> </td>
        <td><input type="button" value="/" onclick="dis('/')"
                        onkeydown="myFunction(event)"> </td>
        </tr>
        <tr>
        <td><input type="button" value="4" onclick="dis('4')"
                        onkeydown="myFunction(event)"> </td>
        <td><input type="button" value="5" onclick="dis('5')"
                        onkeydown="myFunction(event)"> </td>
        <td><input type="button" value="6" onclick="dis('6')"
                        onkeydown="myFunction(event)"> </td>
        <td><input type="button" value="*" onclick="dis('*')"
                        onkeydown="myFunction(event)"> </td>
        </tr>
        <tr>
        <td><input type="button" value="7" onclick="dis('7')"
                        onkeydown="myFunction(event)"> </td>
        <td><input type="button" value="8" onclick="dis('8')"
                        onkeydown="myFunction(event)"> </td>
        <td><input type="button" value="9" onclick="dis('9')"
                        onkeydown="myFunction(event)"> </td>
        <td><input type="button" value="-" onclick="dis('-')"
                        onkeydown="myFunction(event)"> </td>
        </tr>
        <tr>
        <td><input type="button" value="0" onclick="dis('0')"
                        onkeydown="myFunction(event)"> </td>
        <td><input type="button" value="." onclick="dis('.')"
                        onkeydown="myFunction(event)"> </td>

        <!-- solve function call function solve to evaluate value -->
        <td><input type="button" value="=" onclick="solve()"> </td>

        <td><input type="button" value="+" onclick="dis('+')"
                        onkeydown="myFunction(event)"> </td>
        </tr>
        </table>

        <script>

        // Function that display value
```

```
        function dis(val) {
        document.getElementById("result").value += val
        }

        function myFunction(event) {
        if (event.key == '0' || event.key == '1'
                || event.key == '2' || event.key == '3'
                || event.key == '4' || event.key == '5'
                || event.key == '6' || event.key == '7'
                || event.key == '8' || event.key == '9'
                || event.key == '+' || event.key == '-'
                || event.key == '*' || event.key == '/')
                document.getElementById("result").value += event.key;
        }

        var cal = document.getElementById("calcu");
        cal.onkeyup = function (event) {
        if (event.keyCode === 13) {
                console.log("Enter");
                let x = document.getElementById("result").value
                console.log(x);
                solve();
        }
        }

        // Function that evaluates the digit and return result
        function solve() {
        let x = document.getElementById("result").value
        let y = math.evaluate(x)
        document.getElementById("result").value = y
        }

        // Function that clear the display
        function clr() {
        document.getElementById("result").value = ""
        }
        </script>
</body>

</html>
```

The code above implements an interactive calculator as can be seen in:
https://www.geeksforgeeks.org/javascript-calculator/
And the calculator looks like this:



Question 1: The code is currently implemented in HTML and JavaScript. Implement the code using Yew and Rust. Both mouse click and keyboard buttons should work. You can use the Rust crate "meval" to calculate strings as arithmetic expressions.

DEMO this deliverable to the lab instructor.

Question 2: Currently the calculator can add (+), subtract (-), multiply (*), and divide (/). Add three more functional buttons to the calculator which do the following:

- Calculate the exponent: ^
- Calculate the modulo: %
- Calculate the ln (natural logarithm): ln
    - For the log function, when you press the button, it should take everything that is currently on the calculator screen as input.
    - The keyboard key that you will assign to ln will be "L".

Question 3: Add a banner (i.e. a picture) above the calculator.

DEMO this deliverable to the lab instructor.